

Thuật toán độ chính xác cao để tính $N!$

Zhang Yifei

Nguồn gốc: High-precision algorithms for computing $N!$

IOI China candidate team papers / 2001 / Zhang Yifei.pdf

Contents

1	Dẫn nhập	1
2	Về độ chính xác cao	1
2.1	Các kiểu số nguyên chuẩn trong Pascal	1
2.2	Tư tưởng cơ bản của thuật toán độ chính xác cao	1
3	Lựa chọn cấu trúc dữ liệu	2
3.1	Giữ càng nhiều chữ số trong mỗi đoạn càng tốt	2
3.2	Dùng kiểu Comp	2
3.3	Biểu diễn nhị phân	2
4	Tối ưu hóa thuật toán	2
4.1	Độ phức tạp của phép nhân độ chính xác cao	2
4.2	Độ phức tạp của phép nhân liên tiếp	3
4.3	Đặt bộ đệm	3
4.4	Kích thước bộ đệm	4
5	Tính giai thừa bằng phân tích thừa số nguyên tố	4
6	Lũy thừa nhị phân	5
6.1	Cải tiến phép bình phương	5
7	Điều chỉnh sau khi phân tích thừa số nguyên tố	6
7.1	Vì sao cần điều chỉnh	6
7.2	Khi nào nên điều chỉnh	6
8	Kết luận	6

1 Dẫn nhập

Các thuật toán trong bài này chủ yếu nhắm tới ngôn ngữ Pascal. Bài viết đặt ra ba câu hỏi: ta hiểu gì về số học độ chính xác cao, ta đã từng dùng những cấu trúc dữ liệu nào cho số nguyên lớn, và ta đã suy nghĩ kỹ về cách tối ưu thuật toán hay chưa. Với bài toán tính $N!$, ta sẽ thấy một số cách cải thiện có thể làm chương trình nhanh hơn nhiều lần.

2 Về độ chính xác cao

2.1 Các kiểu số nguyên chuẩn trong Pascal

Pascal cung cấp một số kiểu số nguyên chuẩn với miền giá trị cố định:

Kiểu dữ liệu	Miền giá trị	
Shortint	-128	đến 127
Byte	0	đến 255
Integer	-32768	đến 32767
Word	0	đến 65535
Longint	-2147483648	đến 2147483647
Comp	$-9.2 \cdot 10^{18}$	đến $9.2 \cdot 10^{18}$.

Tuy Comp thuộc nhóm kiểu thực, trên thực tế nó biểu diễn một số nguyên 64 bit.

2.2 Tư tưởng cơ bản của thuật toán độ chính xác cao

Các kiểu số nguyên chuẩn trong Pascal nhiều nhất chỉ xử lý trực tiếp được những số nguyên nằm trong khoảng xấp xỉ từ -2^{63} đến 2^{63} . Khi cần tính với số nguyên lớn hơn, ta phải dùng thuật toán độ chính xác cao.

Tư tưởng cơ bản là chia số nguyên lớn, vốn không thể xử lý trực tiếp, thành nhiều đoạn số nguyên nhỏ có thể xử lý trực tiếp. Khi đó phép toán trên số lớn được chuyển thành các phép toán trên những đoạn nhỏ ấy.

3 Lựa chọn cấu trúc dữ liệu

3.1 Giữ càng nhiều chữ số trong mỗi đoạn càng tốt

Xét ví dụ tính tổng của hai số có 15 chữ số. Có ba cách chia số:

- Cách thứ nhất: chia thành 15 đoạn nhỏ, mỗi đoạn có 1 chữ số. Ta cần 15 phép cộng một chữ số.
- Cách thứ hai: chia thành 5 đoạn nhỏ, mỗi đoạn có 3 chữ số. Ta cần 5 phép cộng ba chữ số.
- Cách thứ ba: kiểu Comp có thể xử lý trực tiếp số nguyên 15 chữ số, nên chỉ cần 1 phép cộng.

Khi dùng Integer, cộng hai số một chữ số và cộng hai số ba chữ số có tốc độ như nhau, vì vậy cách thứ hai hiệu quả hơn cách thứ nhất. Dù thao tác trên Comp chậm hơn thao tác trên Integer, số lần cộng giảm rất mạnh; thực nghiệm cho thấy cách thứ ba vẫn nhanh hơn cách thứ hai.

3.2 Dùng kiểu Comp

Trong tính toán độ chính xác cao, mỗi đoạn nhỏ có thể được biểu diễn bằng kiểu Comp. Kiểu này có khoảng 19 đến 20 chữ số có nghĩa. Tùy phép toán, ta chọn số chữ số lưu trong mỗi đoạn như sau:

- Khi cộng hai số độ chính xác cao, mỗi đoạn có thể giữ 17 chữ số.
- Khi nhân một số độ chính xác cao với một số nguyên không quá m chữ số, mỗi đoạn có thể giữ $18 - m$ chữ số.
- Khi nhân hai số độ chính xác cao, mỗi đoạn có thể giữ 9 chữ số.

Nếu mỗi đoạn nhỏ giữ k chữ số thập phân, ta có thể xem đoạn đó là một chữ số trong hệ cơ số 10^k . Trong bài này, ta gọi cách biểu diễn ấy là **số cơ số cao**, và gọi một chữ số cơ số cao là một **số đơn chính xác**.

3.3 Biểu diễn nhị phân

Dùng biểu diễn nhị phân có thể cải thiện hiệu quả thời gian và không gian trong quá trình tính toán. Tuy nhiên, đề bài thường yêu cầu in kết quả ở hệ thập phân, nên cần thêm một bước đổi cơ số rất tốn thời gian. Vì thế phương pháp này thường không được dùng trong thi đấu, và bài viết không đi sâu vào cách làm đó.

4 Tối ưu hóa thuật toán

Các phần sau phân tích độ phức tạp của phép nhân độ chính xác cao, phép nhân liên tiếp, kỹ thuật đặt bộ đệm, cách tính giai thừa bằng phân tích thừa số nguyên tố, phương pháp lũy thừa nhị phân, và bước điều chỉnh sau khi phân tích thừa số nguyên tố.

4.1 Độ phức tạp của phép nhân độ chính xác cao

Khi nhân một số có n chữ số cơ số cao với một số có m chữ số cơ số cao, thuật toán nhân thông thường cần nm phép nhân giữa các chữ số cơ số cao. Kết quả có thể có $n + m - 1$ hoặc $n + m$ chữ số cơ số cao.

4.2 Độ phức tạp của phép nhân liên tiếp

Xét ví dụ tính

$$5 \cdot 6 \cdot 7 \cdot 8.$$

Nếu nhân theo thứ tự từ trái sang phải, ta có

$$\begin{array}{ll} 5 \cdot 6 = 30, & 1 \cdot 1 = 1 \text{ phép nhân,} \\ 30 \cdot 7 = 210, & 2 \cdot 1 = 2 \text{ phép nhân,} \\ 210 \cdot 8 = 1680, & 3 \cdot 1 = 3 \text{ phép nhân.} \end{array}$$

Tổng cộng có 6 phép nhân. Nếu nhóm khác đi,

$$\begin{array}{ll} 5 \cdot 6 = 30, & 1 \cdot 1 = 1 \text{ phép nhân,} \\ 7 \cdot 8 = 56, & 1 \cdot 1 = 1 \text{ phép nhân,} \\ 30 \cdot 56 = 1680, & 2 \cdot 2 = 4 \text{ phép nhân,} \end{array}$$

tổng số phép nhân vẫn là 6.

Đặc điểm của ví dụ này là khi một số n chữ số nhân với một số m chữ số, kết quả có đúng $n + m$ chữ số. Nếu giả thiết ấy luôn đúng, thì với n số đơn chính xác, bất kể chọn thứ tự nhân như thế nào, số phép nhân luôn là

$$\frac{n(n-1)}{2}.$$

Chứng minh bằng quy nạp như sau. Gọi $F(n)$ là số phép nhân cần dùng. Ta có $F(1) = 0$. Giả sử với mọi $k < n$, $F(k) = k(k-1)/2$. Ở lần nhân cuối, ta nhân một số k chữ số với một số $n-k$ chữ số, nên

$$\begin{aligned} F(n) &= F(k) + F(n-k) + k(n-k) \\ &= \frac{k(k-1)}{2} + \frac{(n-k)(n-k-1)}{2} + k(n-k) \\ &= \frac{n(n-1)}{2}. \end{aligned}$$

Giá trị này không phụ thuộc vào cách chọn k .

4.3 Đặt bộ đệm

Trường hợp kết quả của phép nhân có thể chỉ có $n + m - 1$ chữ số cho thấy thứ tự nhân bắt đầu có ảnh hưởng. Ví dụ tính

$$9 \cdot 8 \cdot 3 \cdot 2.$$

Nếu nhân theo thứ tự từ trái sang phải:

$$\begin{aligned} 9 \cdot 8 &= 72, & 1 \cdot 1 &= 1 \text{ phép nhân,} \\ 72 \cdot 3 &= 216, & 2 \cdot 1 &= 2 \text{ phép nhân,} \\ 216 \cdot 2 &= 432, & 3 \cdot 1 &= 3 \text{ phép nhân.} \end{aligned}$$

Tổng cộng có 6 phép nhân. Nếu trước hết tính $9 \cdot 8$ và $3 \cdot 2$, rồi mới nhân hai kết quả:

$$\begin{aligned} 9 \cdot 8 &= 72, & 1 \cdot 1 &= 1 \text{ phép nhân,} \\ 3 \cdot 2 &= 6, & 1 \cdot 1 &= 1 \text{ phép nhân,} \\ 72 \cdot 6 &= 432, & 2 \cdot 1 &= 2 \text{ phép nhân,} \end{aligned}$$

tổng cộng chỉ còn 4 phép nhân.

Ta xét tích của $k + t$ số đơn chính xác

$$a_1 a_2 \cdots a_k a_{k+1} \cdots a_{k+t}.$$

Giả sử $a_1 a_2 \cdots a_k$ đã được tính và kết quả là một số có m chữ số cơ số cao; đồng thời $a_{k+1} \cdots a_{k+t}$ vẫn là một số đơn chính xác. Nếu tiếp tục nhân tuần tự, ta cần t phép nhân kiểu “số m chữ số nhân với số 1 chữ số”, tức khoảng mt phép nhân giữa chữ số cơ số cao. Nếu trước hết tính $a_{k+1} \cdots a_{k+t}$, rồi nhân một lần với kết quả trước, ta chỉ cần khoảng $m + t$ phép nhân.

Trong cách cài đặt có bộ đệm, ta gom nhiều số đơn chính xác lại cho đến khi tích của chúng sắp vượt quá khả năng chứa của kiểu chuẩn, rồi mới nhân tích đó vào số độ chính xác cao. Khi tính tích của m số đơn chính xác và kết quả cuối cùng có n chữ số cơ số cao, số phép nhân xấp xỉ

$$\frac{n(n-1)}{2},$$

gần như không phụ thuộc trực tiếp vào m . Có thể hình dung ta chia m thừa số thành n nhóm, mỗi nhóm có tích vẫn là một số đơn chính xác; phần tồn thời gian chính là nhân n số thu được. Lúc đó mô hình “ n chữ số nhân m chữ số cho $n + m$ chữ số” gần đúng, nên công thức trên áp dụng được.

4.4 Kích thước bộ đệm

Giả sử kiểu dữ liệu chuẩn được chọn có thể xử lý trực tiếp tối đa t chữ số thập phân. Đặt kích thước bộ đệm là k chữ số thập phân; khi đó mỗi đoạn nhỏ của số độ chính xác cao giữ $t - k$ chữ số thập phân. Nếu kết quả cuối cùng có n chữ số thập phân, thì số phép nhân xấp xỉ

$$\frac{k}{n-k} \sum_{i=1}^{n/k} i = \frac{(n+k)n}{2k(t-k)},$$

trong đó k nhỏ hơn n rất nhiều.

Để số phép nhân nhỏ nhất, ta cần làm cho $k(t-k)$ lớn nhất. Giá trị tối ưu đạt được khi $k = t/2$. Vì vậy, hiệu quả cao nhất xuất hiện khi kích thước bộ đệm bằng kích thước mỗi đoạn nhỏ. Trong phép nhân liên tiếp thông thường, có thể dùng Comp làm kiểu cơ sở, mỗi đoạn nhỏ giữ 9 chữ số thập phân, và bộ đệm cũng giữ 9 chữ số thập phân.

5 Tính giai thừa bằng phân tích thừa số nguyên tố

Ví dụ:

$$10! = 2^8 \cdot 3^4 \cdot 5^2 \cdot 7.$$

Độ phức tạp của việc phân tích $n!$ thành thừa số nguyên tố nhỏ hơn rất nhiều so với $n \log n$, nên có thể xem là không đáng kể so với phần nhân độ chính xác cao.

So với thuật toán thông thường, sau khi phân tích thừa số nguyên tố, số lượng thừa số m có thể tăng, nhưng số chữ số n của kết quả không đổi. Miễn là dùng bộ đệm, số phép nhân vẫn xấp xỉ $n(n-1)/2$. Vì vậy, riêng bước phân tích thừa số nguyên tố không làm thuật toán chậm đi; điều này cũng phù hợp với thực nghiệm.

Lợi ích của việc phân tích thừa số nguyên tố nằm ở chỗ sau đó xuất hiện rất nhiều phép tính lũy thừa. Ta có thể tối ưu phần tính lũy thừa, và nhờ vậy phương pháp phân tích thừa số nguyên tố mới phát huy hiệu quả.

Một quy trình tính $N!$ theo hướng này là:

1. Liệt kê các số nguyên tố $p \leq N$.
2. Với mỗi p , tính số mũ

$$e_p = \sum_{i \geq 1} \left\lfloor \frac{N}{p^i} \right\rfloor.$$

3. Tính các lũy thừa p^{e_p} bằng phương pháp lũy thừa nhị phân, dùng phép bình phương đã cải tiến.
4. Điều chỉnh và ghép các lũy thừa khi có lợi, rồi nhân các khối kết quả bằng phép nhân độ chính xác cao có bộ đệm.

6 Lũy thừa nhị phân

Lũy thừa nhị phân dùng các công thức

$$a^{2n+1} = a^{2n} \cdot a, \quad a^{2n} = (a^n)^2,$$

trong đó a là một số đơn chính xác.

Giả sử tích của một số n chữ số với một số m chữ số luôn có $n + m$ chữ số. Gọi $F(n)$ là số phép nhân giữa chữ số cơ số cao cần dùng để tính a^n bằng lũy thừa nhị phân. Khi đó

$$F(2n) = F(n) + n^2, \quad F(1) = 0.$$

Nếu $n = 2^k$, ta có

$$F(n) = F(2^k) = \sum_{i=0}^{k-1} 4^i = \frac{4^k - 1}{3} = \frac{n^2 - 1}{3}.$$

Trong khi đó, nhân liên tiếp cần $n(n-1)/2$ phép nhân. Như vậy nhân liên tiếp chỉ nhiều hơn lũy thừa nhị phân khoảng 50%. Việc dùng lũy thừa nhị phân không làm độ phức tạp giảm từ n^2 xuống $n \log n$, vì chi phí của các phép bình phương số lớn vẫn chi phối.

6.1 Cải tiến phép bình phương

Ta khai thác đẳng thức

$$(a + b)^2 = a^2 + 2ab + b^2.$$

Chẳng hạn,

$$12345^2 = 123^2 \cdot 10000 + 45^2 + 2 \cdot 123 \cdot 45 \cdot 100.$$

Ý tưởng là tách một số n chữ số thành một phần t chữ số và một phần $n - t$ chữ số, rồi tính bình phương bằng công thức trên.

Gọi $F(n)$ là số phép nhân cần dùng để bình phương một số n chữ số. Ta có

$$F(n) = F(t) + F(n-t) + t(n-t), \quad F(1) = 1.$$

Bằng quy nạp toán học, suy ra

$$F(n) = \frac{n(n+1)}{2}.$$

Vì thế, bất kể chia ở vị trí nào, hiệu quả về số phép nhân là như nhau. Trong cài đặt, tách thành một phần 1 chữ số và một phần $n - 1$ chữ số là thuận tiện hơn.

Cũng có thể nhìn phép bình phương theo công thức

$$S^2 = \left(\sum_{0 \leq i < n} a_i 10^i \right)^2 = \sum_{0 \leq i < n} a_i^2 10^{2i} + 2 \sum_{0 \leq i < j < n} a_i a_j 10^{i+j}.$$

Tổng cộng có $n + \binom{n}{2} = n(n+1)/2$ phép nhân. Thuật toán bình thường cần n^2 phép nhân, tức chậm hơn khoảng một lần so với phép bình phương đã cải tiến.

Nếu trong lũy thừa nhị phân ta dùng phép bình phương cải tiến, số phép nhân cần dùng trở thành

$$\frac{(n+4)(n-1)}{6},$$

xấp xỉ một phần ba so với thuật toán nhân liên tiếp với $n(n-1)/2$ phép nhân.

7 Điều chỉnh sau khi phân tích thừa số nguyên tố

7.1 Vì sao cần điều chỉnh

Khi tính

$$S = 2^{11} 3^{10},$$

ta có thể tính riêng 2^{11} , tính riêng 3^{10} , rồi nhân hai kết quả. Nhưng cũng có thể dùng

$$S = 2^{11} 3^{10} = 6^{10} \cdot 2,$$

tức tính 6^{10} rồi nhân thêm 2. Hai cách này có hiệu quả khác nhau.

7.2 Khi nào nên điều chỉnh

Xét

$$S = a^{x+k}b^x = (ab)^xa^k.$$

Khi

$$k < x \log_a b,$$

dùng dạng $(ab)^xa^k$ tốt hơn; ngược lại, dùng dạng $a^{x+k}b^x$ nhanh hơn.

Có hai cách hiểu kết luận này. Cách trực tiếp là tính số phép nhân của hai phương án rồi giải bất đẳng thức. Một cách nhìn khác là: khác biệt chính của hai phương án nằm ở phép nhân cuối cùng. Vì tích cuối cùng của hai cách có cùng số chữ số, hai thừa số càng chênh lệch thì số phép nhân cuối cùng càng ít.

Cụ thể, ta chọn $(ab)^xa^k$ khi

$$a^xb^x - a^k > a^{x+k} - b^x.$$

Bất đẳng thức này tương đương với

$$(b^x - a^k)(a^x + 1) > 0,$$

nên tương đương với

$$b^x > a^k.$$

Do đó điều kiện là

$$k < x \log_a b.$$

8 Kết luận

Những điểm chính của bài viết là:

- dùng Comp làm kiểu cơ sở cho mỗi đoạn nhỏ;
- cân nhắc biểu diễn nhị phân, dù trong thi đấu thường phải trả giá ở bước đổi sang hệ thập phân;
- dùng bộ đệm khi nhân liên tiếp độ chính xác cao, và chọn kích thước bộ đệm bằng kích thước mỗi đoạn nhỏ;
- dùng phép bình phương cải tiến;
- dùng lũy thừa nhị phân;
- tính giai thừa bằng phân tích thừa số nguyên tố, kèm điều chỉnh sau khi phân tích.

Các kỹ thuật này có thể dùng cho tính lũy thừa độ chính xác cao, tính tích liên tiếp như giai thừa, và tính số chính hợp hoặc tổ hợp độ chính xác cao.

Tự thân bài toán tính $N!$ bằng độ chính xác cao không khó, nhưng vẫn có thể được tối ưu từ nhiều góc độ. Thật ra, nhiều thuật toán kinh điển cũng còn không gian cải thiện, và những chương trình do ta tự viết cũng vậy. Chỉ cần chịu khó phân tích, rất có thể ta sẽ tìm ra một thuật toán tốt hơn. Trong thi đấu, yêu cầu về độ chính xác cao thường không quá nặng, nên các tối ưu này có thể không bắt buộc; mục tiêu chính của ví dụ này là nhấn mạnh một ý: thuật toán luôn có thể được tối ưu.